Helm Tutorial with Examples & Interview Q&A

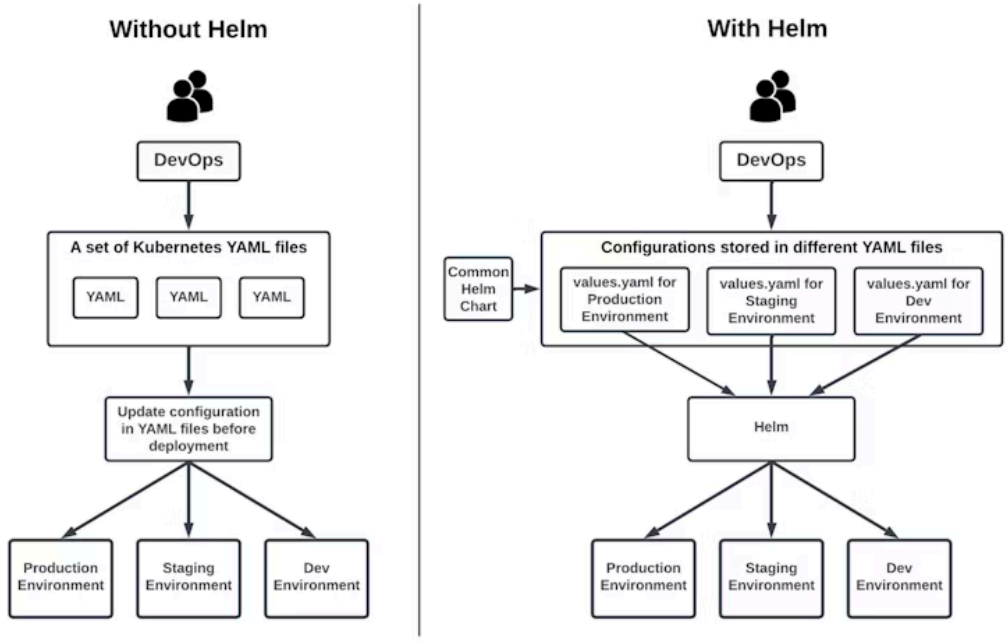
What is Helm?

Helm is a **package manager** for Kubernetes, similar to how apt or yum work for Linux. Helm helps you define, install, and upgrade Kubernetes applications using Helm Charts.

Key Concepts

- **Chart**: A Helm package containing all necessary Kubernetes resource definitions.
- **Release**: A specific deployment of a chart in a Kubernetes cluster.
- **Repository**: A place where charts are stored and shared.

Kubernetes Deployments: Without Helm vs With Helm



Without Helm

- DevOps engineers manually maintain multiple raw Kubernetes YAML files.
- Configuration values are manually edited before each deployment.
- This process is repetitive, error-prone, and hard to maintain across multiple environments (Dev, Staging, Production).

With Helm

- Helm uses **templated charts** and **environment-specific values.yaml** files.
- A **single chart** can be reused across environments, reducing duplication.
- Helm injects the correct values into the templates at deploy time.
- Makes deployments faster, consistent, scalable, and easier to version/control.

Helm Architecture

- **Helm CLI**: Used by the user to interact with Helm.
- **Tiller (Helm v2 only)**: A server component that manages releases (removed in Helm v3).
- **Kubernetes API Server**: Where Helm applies the manifests.

Helm Chart Structure

```
mychart/
├── Chart.yaml       # Chart metadata
├── values.yaml      # Default configuration values
├── charts/          # Dependency charts
├── templates/       # Kubernetes manifest templates
```

Sample Chart.yaml

```
apiVersion: v2
name: my-nginx
version: 0.1.0
description: A Helm chart for Kubernetes
```

Sample values.yaml

```
replicaCount: 2
image:
  repository: nginx
  tag: latest
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
```

Sample deployment.yaml template

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-nginx.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "my-nginx.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "my-nginx.name" . }}
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

Common Helm Commands and Their Uses

Command	Purpose
helm repo add <name> <url>	Add a Helm chart repository
helm search repo <keyword>	Search charts in repositories
helm install <release-name> <chart>	Install a chart as a release
helm upgrade <release-name> <chart>	Upgrade an existing release
helm uninstall <release-name>	Uninstall a release
helm list	List all Helm releases
helm show values <chart>	View default values of a chart
helm lint <chart-dir>	Check chart for formatting and best practices
helm template <chart-dir>	Render chart templates locally
helm install --dry-run --debug <release-name> <chart>	Simulate an install to debug issues

Best Practices

- Use _helpers.tpl for reusable templates.
- Externalize configurations to values.yaml.
- Use helm lint to validate your chart.
- Perform dry runs using helm install --dry-run --debug.

Helm with GitOps

Helm works well with GitOps tools like ArgoCD and Flux. Store your Helm charts in Git repositories. The GitOps tool continuously syncs the charts to your Kubernetes cluster.

Interview Questions & Answers

Q1: What is Helm?

A: Helm is a package manager for Kubernetes that simplifies deployment using Helm Charts.

Q2: What is the difference between Helm v2 and v3?

A: Helm v3 removed the server-side component Tiller, improving security and making Helm more Kubernetes-native.

Q3: What is a Helm Chart?

A: A Helm Chart is a collection of YAML templates and configuration files that define a Kubernetes application.

Q4: How do you upgrade a Helm release?

A: Use **helm upgrade <release-name> <chart-path>** to upgrade an existing deployment.

Q5: What are values.yaml and templates used for?

A: values.yaml provides default configurations. Templates generate the Kubernetes manifests using those values.

Q6: What is a release in Helm?

A: A release is an instance of a chart running in a Kubernetes cluster.

Q7: How do you override values during installation?

A: Use **-f custom-values.yaml** or **--set key=value** while installing or upgrading.

Q8: How does Helm support CI/CD?

A: Helm is used in CI/CD pipelines to package and deploy applications consistently. It works with tools like Jenkins, Azure DevOps, and GitHub Actions.

Q9: How do you debug Helm chart issues?

A: Use:

- helm lint
- helm template
- helm install --dry-run --debug

Q10: What is the use of _helpers.tpl?

A: _helpers.tpl contains reusable named template definitions that help avoid repetition in templates.